Code Lab 2

Burcu Erarslanoglu (5948355)

EE4C12: Machine Learning for Electrical Engineering Applications
Delft University of Technology

September 18, 2024

1 Task 1: Load Energy Data

In this part the file containing all the related information is loaded and read first using the Pandas library. The columns carry information about day, month, hours, precipitation, temperature, irradiance_surface, irradiance_toa, snowfall, snow_mass, cloud_cover, air_density, Forecast load (MW), Actual load (MW), Positive imbalance price (Eur/MWh) and Negative imbalance price (Eur/MWh).

After the data is loaded, the date and hour corresponding to the maximum electricity consumption (Actual load with unit MW) are found as below:

```
The date for maximum electricity consumption: 22/1
The hour for maximum electricity consumption:  17
```

Next, for the date of maximum electricity consumption which is 22^{nd} of January, some variables are plotted for the entire day. This plot can be found below:

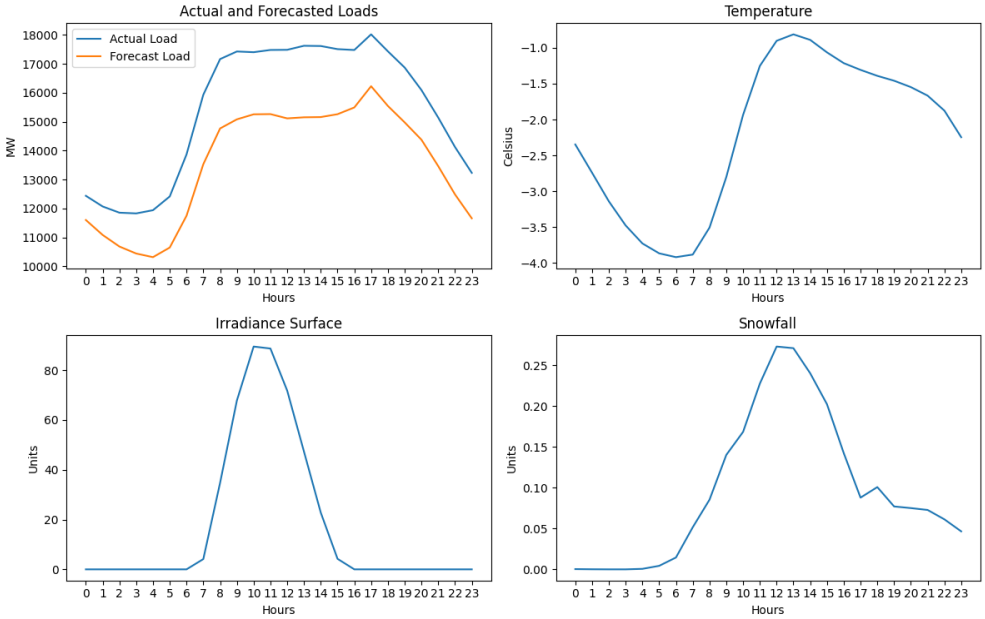

Figure 1: Data for the Day of Maximum Electricity Consumption.

After this visualization, the lab proceeds with the introduction of three different feature matrices X_1, X_2, and X_3. The data of these feature matrices are loaded by using the pickle command from the dictionary "D". The keys of this dictionary are then printed as follows:

```
Keys: dict_keys(['Feature1', 'Feature2', 'Categorical', 'Target'])
```

As it can be seen there are 3 features and 1 target variable. The contents of this dictionary are extracted to the variables which have the names "X_1", "X_2", "C" and "y" respectively.

The last step of this part is finding the sparsity of "X_1", "X_2", "C", which is found below:

```
Sparsity of X_1:  0.2159880389628993
Sparsity of X_2:  0.20056032189412076
Sparsity of C:  0.6666666666666666
```

Now, the questions about Task 1 can be answered.

Question 1

This question requires a bit of coding and as a result, its answer is found below:

```
Difference between the forecasted and actual loads at the hour of the maximum
    consumption: 1792.5 MW
```

Question 2

According to fig. 1, we can observe a clear relationship between environmental factors and energy consumption throughout the day. My first observation is that the forecast consistently underestimates the actual load. The actual electricity load peaks in the late afternoon around 17:00–18:00. The peaks for the actual and forecasted loads most likely coincide with increased activity and higher energy demands. Temperature during the day is also influential, as it is lowest in the early morning and rises until the early afternoon, influencing heating needs during colder hours. Solar irradiance of the surface peaks around midday, reflecting potential solar energy generation. On the other hand, snowfall peaks later, potentially increasing heating demands as well. The combination of lower temperatures, lower irradiance during peak consumption hours, and snowfall suggests that energy consumption is driven by both heating needs and decreased availability of energy sources, such as solar power by the sun. All of these factors can help to explain the day's high electricity consumption.

Question 3

From the first subplot, showing the actual and forecasted loads, it can be derived that the temperature, shown in the second subplot, has a big influence. The shape of the temperature over the day can be found in the actual load. The irradiance surface does not seem to have a noticeable effect on the actual load. It has a high peak over a specific time range, which does not show an impact on the actual forecast. For snowfall, it seems to be the same as it is hard to find a correlation between it and the actual load. But, it would be expected that the snowfall would affect power usage. Other important features are the time and day of the week, as usage is very dependent on work hours and people's daily activities. Overall, firstly temperature and then snowfall are good features for a machine learning model.

Question 4

The sparsity of "C" was found before as:

```
Sparsity of C:  0.6666666666666666
```

Question 5

Encoding for categorical features such as date or time is necessary for machine learning (ML) models because ML models struggle with interpreting raw numeric representations of dates. For example, when a numeric value for a day is entered, this causes the last weeks of the month to have a larger impact on prediction. This undeniably affects the prediction capabilities of the ML model. In our

example, one-hot encoding was used to avoid false ordinal relationships by representing categories in a binary format. In this format, each month or day of the week can be represented by a separate binary feature as it is shown in detail for the Bonus Task. This type of encoding makes sure that every category is treated independently.

Question 6

For the forecasting of energy load, time information is required because electricity demands change daily and monthly. The seasons also contribute to this change. For example, energy consumption could peak at homes in the mornings and evenings because of increased human activity. On the other hand, energy consumption could peak at industrial sites during the day. Weekdays and weekends exhibit different loads as well. Another important factor is the seasons where the heating and cooling requirements change drastically. Therefore, including time information allows the model to capture these natural cyclical patterns that are present in everyday life. This improves the ability of the ML model to predict when spikes or dips in consumption occur.

Question 7

First of all, let's start with writing down what sparsity is. Sparsity refers to the proportion of zero values in a matrix. With this definition, we can see how the matrix "C" has a more "sparse" nature compared to "X_1" and "X_2". This is due to the fact that "C" contains one-hot encoded categorical features, where certain features are rarely active. High sparsity occurs when most elements in the matrix are zero. This could be inefficient in terms of storage but this also means that the model has less information to learn from concerning certain categories. On the other hand, low sparsity means that most elements in the matrix are non-zero. This means that the ML model has more information to work with, where it has better learning and modeling performance. Overall, calculating sparsity is important since it provides valuable insights into how the ML model works and learns. In terms of its effect on regression; if sparsity is high some regularization techniques might be needed to prevent the model from over-fitting onto the few non-zero values. If sparsity is low, the regression process has more data points to learn from but it needs to handle the vast amounts of data more carefully.

2 Task 2: Process Data and Prepare for Machine Learning Algorithms

This task begins with standardizing noncategorical variables "X_1" and "X_2". This is done by removing the mean and scaling the unit variance. "X_1" and "X_2" are scaled by using different scalers. Later, the categorical variable matrix C is horizontally concatenated with the scaled X_2 to generate the third feature set X_3. No output was wanted for this part but I looked at the shapes of these matrices and printed them as below:

```
X_1 scaled shape: (8592, 13)
X_2 scaled shape: (8592, 14)
C shape: (8592, 9)
X_3 shape: (8592, 23)
```

The next part of this task is to split the datasets into training and test sets where 25% of the data is used for test. Shuffle_state variable is used as the random state to have the same y_train and y_test across different splits. The splitting of X_1, X_2, and X_3 gives rise to 3 training and test data sets. The outputs wanted for this part are presented below:

```
Mean of X_1 = 5030.273
Mean of scaled X_1 = 0.000
```

```
Mean of scaled X_train1 = -0.001
Mean of scaled X_test1 = 0.004
```

These results make sense because the unscaled X_1 has a high mean where the raw data is used. The mean of the scaled X_1 is 0 as expected because when the scaler centers the data around 0 by subtracting the mean of the original data. The mean of the scaled X_{train} is very close to zero but has a small difference because the splitting introduces a small difference between the training and the entire dataset. It is normal to have a small numerical distance after the split for both X_{train} and X_{test} because of the random splitting.

Now, the questions about Task 2 can be answered.

Question 1

When two different scalars are used, we ensure that each feature set is scaled according to the statistical characteristics of the data in that feature set. Because the scaling process standardizes the data by removing its mean and scaling it to unit variance, different scalers should be used for datasets with different means and variances. As a result of this process, X_1 and X_2 are centered and scaled independently and the relationships within each set are preserved.

Question 2

Categorical variables are discrete categories usually having boolean values after one-hot encoding. Before one-hot encoding, they represent categorical variables such as the colors of red and blue. Therefore, they are not continuous values. Scaling these variables wouldn't produce meaningful results because they are not in a numerical range where they get their values. Because of this, categorical variables are encoded, and then these encoded variables are used in ML models.

3 Task 3: Training and Evaluation of Linear Regressor

In this part, different regression models are implemented on the feature sets X_1, X_2 (both scaled) and X_3.

First, three linear regression models are constructed and trained for feature sets X_1, X_2, X_3.

Then a function called test_mymodel(model,X_test,y_test) is written to evaluate the model performance by the metrics of R^2 score and mean square error (MSE). This function is provided below:

```
#Model evaluation function
def test_mymodel(model,X_test,y_test):
    y_prediction = model.predict(X_test) # Use X_test to predict y_test as
    y_prediction
    MSE = mean_squared_error(y_test, y_prediction)
    R2 = r2_score(y_test, y_prediction)
    return MSE,R2
```

Next, the linear regression models are evaluated with this function, and the following outputs are produced:

```
MSE scores of linear regressors: 928883.96, 622490.38, 421009.82
R2 scores of linear regressors: 0.7511760412272825, 0.8332509462161521,
    0.8872223706802830
```

These results can also be visualized in bar graphs as shown below:

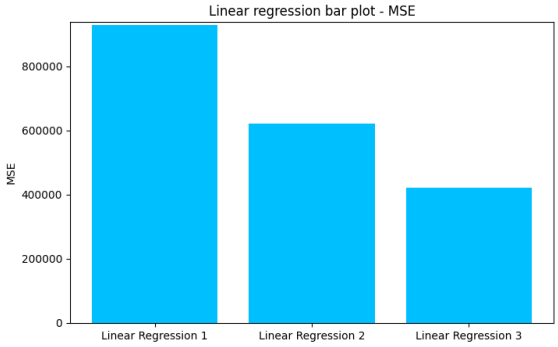

Figure 2: MSE for the three linear regression models.

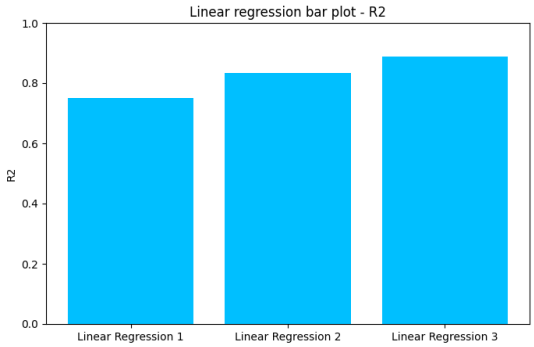

Figure 3: R^2 score for the three linear regression models.

Next, the test data predictions of the three data models are carried out and this test data predictions together with the first 64 sample points is plotted below:

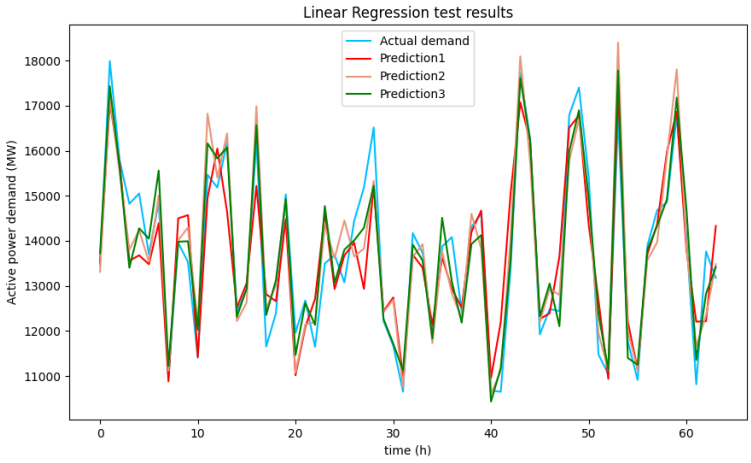

Figure 4: Data prediction using the linear regression models.

According to the bar graphs at fig. 2 and fig. 3 and predictions at fig. 4; the best model is Linear Regressor 3, which was based on X_3. For the choice of the best linear regressor, a higher R^2 score is preferred since such a model explains the variance in the data better. As for the MSE, the lower the better because it is the average of the squared difference between the actual and predicted values meaning that lower values for MSE indicate more accurate predictions.

The model weights of this best model (3) and the absolute maximum difference between its weights can be found below:

```
Coefficients of the best model: [   17.94423218  -326.42386824  -481.63566056
    548.67731475
     25.5450198    -19.84711394    21.16472399  -132.01353121
  1013.44354774 -1043.45228794   165.50478211    96.80661346
   520.20716573   671.84417051  -425.22014807   425.22014807
  -196.66521128   -62.39361706   160.41813932    98.64068903
   291.96236016   525.61856801  -817.58092816]
Absolute max difference = 2056.90
```

In the next part of Task 3, Ridge and Lasso regression models which are the modified forms of linear regression are investigated. Ridge and Lasso regression models use a parameter called "alpha" to control complexity. The "alpha" values in Ridge and Lasso have different effects in training since they manifest themselves differently during the regularization of the optimization problem. Newly scaled versions for the target variable's training and testing data are used in this part to avoid convergence problems. The best feature set X_3 is used in this part since its regressor gave the best results in terms of R^2 score and MSE. The results for Ridge and Lasso in terms of R^2 score and MSE can be found below:

```
MSE scores of ridge regressor: 0.11
MSE scores of lasso regressor: 0.23
R2 scores of ridge regressors: 0.88722
R2 scores of lasso regressors: 0.76927
```

Later, both models are trained and tested with varying "alpha" values, and results are recorded in each step as follows:

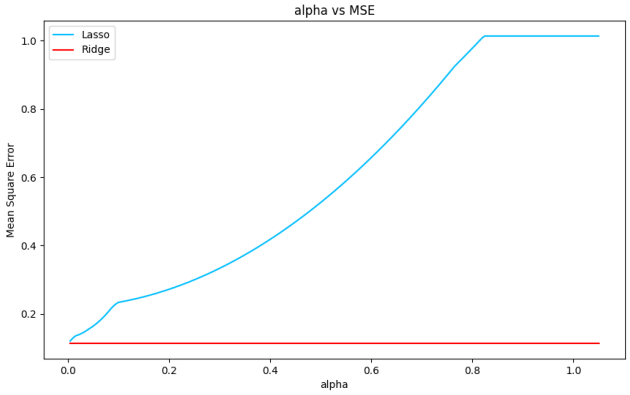

Figure 5: Ridge and Lasso Regressor performances in terms of MSE for varying alpha.

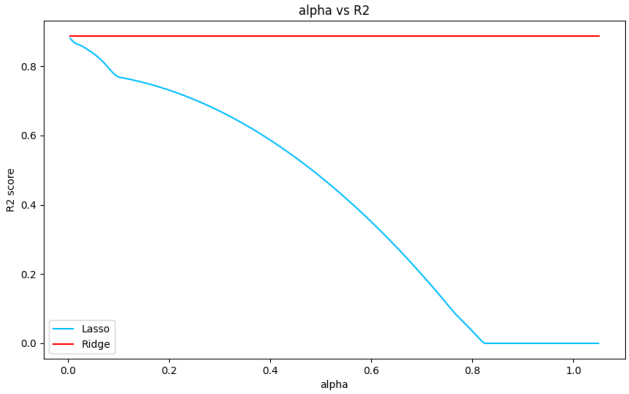

Figure 6: Ridge and Lasso Regressor performances in terms of R^2 score for varying alpha.

Looking at the figures above it is seen that for the Ridge model, the changes in alpha do not change MSE and R^2 score. Therefore, alpha = 0.005 (lowest possible value in the range of alpha) can be considered the best-performing parameter since increasing alpha does maintain the lowest MSE and highest R^2 values. For the Lasso model, increasing trends in MSE and a decreasing trend in R^2 are seen as alpha increases. Therefore, the best-performing value of alpha for Lasso regression appears to be near zero (close to 0.005), where the model achieves the lowest MSE and highest R^2 score. The effect of alpha on prediction for Ridge is non-existent since the regularization already makes sure that the weights are not too big and not zero. Therefore, the weights are probably already small without the regularization parameter alpha. Therefore, adding the regularization parameter does not affect the results. For the Lasso regression, as alpha increases the Lasso model performs worse, leading to higher MSE and lower R^2. Therefore, it has poorer prediction performance for higher alphas.

Question 1

Linear Regression model 1 performs the worst with the highest MSE and lowest R^2 score. Linear Regression model 2 improves in both metrics and Linear Regression model 3 has the best performance. These results are expected as the data of X_3 shows more detailed features and therefore the Linear Regression model can capture more relevant relationships. This also indicates that the model which was based on X_3 explains the variance in the data the best (high R^2 score) and predicts the targeted data more accurately (low MSE score).

Question 2

The Mean Squared Error (MSE) represents the average of the squared differences between the actual values and the predicted values. In other words, it is used to show how well the regression model fits the data. A lower MSE indicates better predictive accuracy as the difference between the actual and predicted values becomes smaller. Since this metric squares the errors, MSE penalizes larger errors more heavily, making it sensitive to outliers. In this CodeLab, MSE represents how far the model's predictions deviate from the actual target values on average.

Question 3

The feature set X_3 gives the best prediction capability to the ML model because it is an extended version of X_2 which was generated by combining the matrices of X_2 (which was an extended version of X_1) and C (which was the categorical variables in a one-hot encoded fashion). This essentially means that X_3 has more detailed features for the ML model to learn from which ultimately increases

its prediction capability.

Question 4

As was also presented before, the absolute maximum difference between the weights of the best-performing model (3):

```
Absolute max difference = 2056.90
```

This is the difference between the maximum and minimum weights.

Question 5

Linear Regression fits a linear model with coefficients. These weights correspond to the values that multiply each of the input features to determine the model's output. They represent the strength (absolute value) and direction (sign) of the relationship between each feature and the prediction. The weights can indicate the importance of each feature in the case of when the data is scaled. When the data is not scaled, the weights cannot directly be used for assessing the importance of each feature.

Question 6

In both Ridge and Lasso regression models, the alpha (α) is a regularization hyperparameter. This hyperparameter controls the strength of the penalty applied to the model's coefficients. In Ridge regression, alpha times the sum of the squared coefficients is the penalty. This process will cause the coefficients to go to close to zero, but not zero. In Lasso regression, you penalize the sum of the absolute values of the coefficient's time alpha. This method can make the coefficients go to exactly zero. When the alpha is low, the model's coefficients are less constrained and will become bigger. This will result in a model which will be fitting the training data more extensively, which can lead to overfitting. When a larger alpha is applied to the model, it will lead to a more generalized model. The discussion on alpha (α) concerning the Ridge and Lasso regression models used in our scenario can be found above (below fig. 6).

Question 7

In linear regression, the model tries to fit the data as closely as possible. When the model is tested on unseen data, big chances are that the model will predict more incorrect in large deviation due to the overfitting nature of Linear Regression. Because of the regularization in Ridge and Lasso, large coefficients are penalized, which leads to a simpler model that generalizes better, thus reducing the MSE. Alpha is set to almost zero (0.005), so there is still some regularization happening and therefore not the same as Linear Regression.

Question 8

The regularized models of Ridge and Lasso, perform better than the linear regressions. This can be understood from their MSE values which are more than a thousand times smaller. For the R^2 score both Ridge and Lasso have higher values than that of the linear regressors. This indicates better accuracy with less overfitting.

Question 9

The value of alpha that gives the smallest error in the test set is very close to zero with a value of 0.005, which was the starting value for the array that contained the alpha values. This is understood by its low MSE value in fig. 5. The high R^2 score for this value fig. 6 is also a bonus since, the variance in the data is explained better for alpha=0.005.

4 Task 4: Support Vector Machines

For this task, a linear Support Vector Machine (SVM) is constructed using default parameters. The recorded performance metrics and the training time can be found below:

```
MSE scores of linear SVM: 0.11654203034659039
R2 scores of linear SVM: 0.8849485318868058
Required Time for linear SVM seconds= 5.585723876953125 seconds
```

Later, a grid search is constructed by using all the possible pairs between the regularization parameter $C = [0.01, 0.1, 1, 5]$ and $\epsilon = [0.01, 0.1, 1, 5]$. The required time for this operation is:

```
Required Time= 60.1499388217926 seconds
```

The best-performing parameters for the case of highest R^2 score are found below:

```
Best R2: 0.8851675329498069
Best C: 1.0499999999999998
Best epsilon: 0.01
```

If we were to choose MSE and specifically the lowest MSE as our performance metric, we would have found different best-performing parameters for this part.

For this part, the default MSE and R^2 score along with the MSE and R^2 score obtained with the best-performing parameters showing highest R^2 score is portrayed below:

```
MSE scores of linear SVM default: 0.11654203034659039
R2 scores of linear SVM default: 0.8849485318868058
MSE scores of linear SVM best: 0.11632019199068955
R2 scores of linear SVM best: 0.8851675329498069
```

The next part constructs a polynomial SVM regressor with the parameters found above. For this part, the outputs are (auto gamma):

```
MSE scores of polynomial SVM: 0.06058033750536053
R2 scores of polynomial SVM: 0.9401944796391781
Required Time for polynomial SVM = 3.274735927581787 seconds
```

When the gamma parameter is adjusted from auto to scale the values change as (scaled gamma):

```
MSE scores of polynomial SVM: 0.051311540869576705
R2 scores of polynomial SVM: 0.9493447291879304
Required Time for polynomial SVM = 5.9678099155426025 seconds
```

Next, an RBF SVM regressor with best-performing hyperparameters is constructed having the following outputs:

```
MSE scores of Gaussian SVM: 0.04343895524224106
R2 scores of Gaussian SVM: 0.9571166251432192
Required Time for Gaussian SVM = 3.3411779403686523 seconds
```

Later, the kernel scale gamma is changed. The time and performance metrics of the optimization problem are given as follows:

```
MSE  0.0709988257154743, 0.0461801543292212, 0.0384327468429510, 0.0372960768597149,
    0.0407028480229167, 0.0471257608969117
R2  0.929909242969379, 0.954410485749477, 0.962058804581024, 0.963180936655972,
    0.959817737793841, 0.953476973405933
Required Time for Gaussian SVM kernel search: 23.48 seconds
```

The plots of gamma vs. R^2 and gamma vs. MSE are presented below:

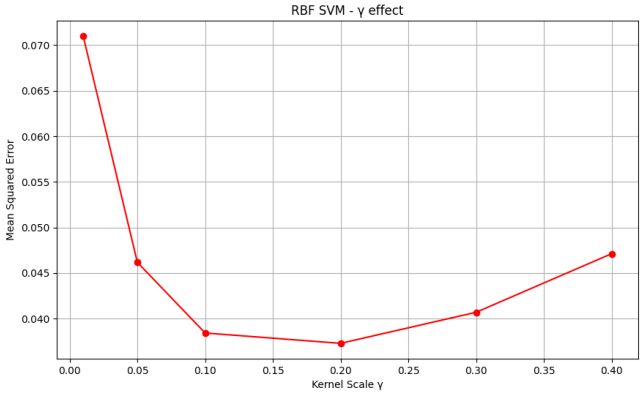

Figure 7: MSE curve for varying gamma.

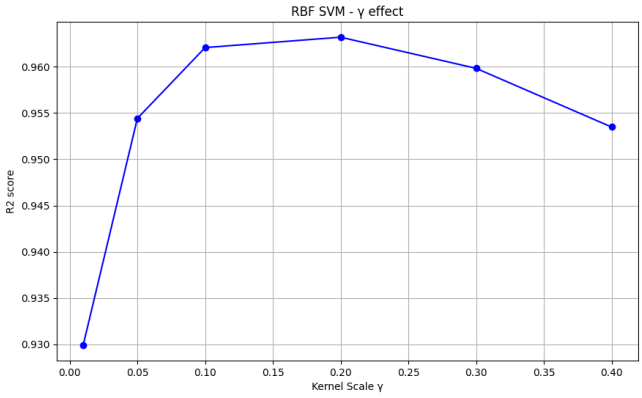

Figure 8: R^2 score curve for varying gamma.

The plot showing the predictions and labels together is presented below:

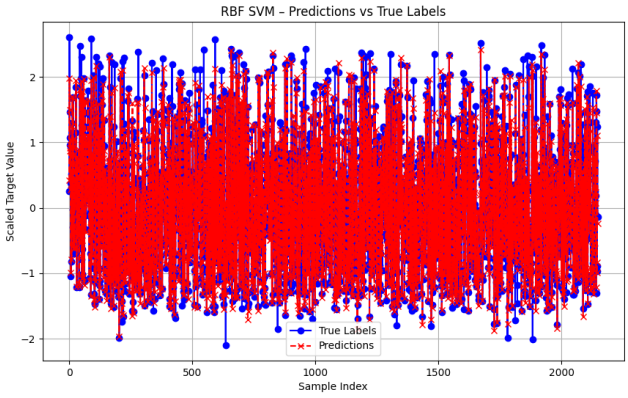

Figure 9: Predictions and true labels.

The fine-tuning of the best SVM model for the regression problem was carried out and the

following results which are slightly better than the ones in the solution manual were found:

```
Best R2 score: 0.97648 (C=18, epsilon=0.015, gamma=0.12)
Best MSE score: 0.02383 (C=18, epsilon=0.015, gamma=0.12)
```

Question 1

Default linear SVM results:

```
MSE scores of linear SVM default: 0.11654203034659039
R2 scores of linear SVM default: 0.8849485318868058
Required Time for linear SVM seconds= 5.585723876953125 seconds
```

Tuned linear SVM results:

```
MSE scores of linear SVM best: 0.11632019199068955
R2 scores of linear SVM best: 0.8851675329498069
Required Time for linear SVM seconds (grid search)= 60.1499388217926 seconds
```

Comparison: The improvement in performance after tuning the SVM was minimal where R^2 score improved slightly and the MSE decreased marginally. The time taken on the other hand increased significantly due to the grid search. This shows the trade-off between computation time and performance improvement.

Question 2

Auto gamma parameter results for polynomial SVM:

```
MSE scores of polynomial SVM: 0.06058033750536053
R2 scores of polynomial SVM: 0.9401944796391781
Required Time for polynomial SVM = 3.274735927581787 seconds
```

Scaled gamma parameter results for polynomial SVM:

```
MSE scores of polynomial SVM: 0.051311540869576705
R2 scores of polynomial SVM: 0.9493447291879304
Required Time for polynomial SVM = 5.9678099155426025 seconds
```

Comparison: As it can be seen, the scaled gamma configuration resulted in better performance where both MSE and R^2 score improved. However, the time required to train the model increased from 3.27 seconds to 5.97 seconds.

Question 3

For this question, fig. 8 and fig. 9 is observed. Gamma is the hyperparameter which defines how far the influence of a single training sample reaches. In other words, a small gamma will make the influence of training points very far-reaching. This causes the decision boundary to be smoother and generalized as it takes more points into account for the generation of the boundary. A large gamma means the opposite, as the influence of each training sample is very local. This causes the model to generalize less which makes the boundary more complex and most likely overfitted. So as can be seen for low gamma values the MSE is high due to the high generalization. Where a high gamma causes the MSE to increase slowly. For the more in-between value 0.20, it has the 'optimal' hyperparameter as it generalizes enough but still makes the boundary complex enough to classify more correctly. For the R^2 score shown in the second plot, the same conclusion is found.

Question 4

The model with the best performer is the RBF SVM regressor when MSE and R^2 scores are considered. This Gaussian SVM gives the following results:

```
MSE scores of Gaussian SVM: 0.04343895524224106
R2 scores of Gaussian SVM: 0.9571166251432192
Required Time for Gaussian SVM = 3.3411779403686523 seconds
```

As can be observed, Gaussian SVM is superior to linear and polynomial SVM since its MSE score is the lowest and R^2 score is the highest. Also, the required time for the Gaussian SVM is comparable to that of the polynomial SVM with the auto gamma parameter which had the smallest overall required time. The parameters for the Gaussian SVM were also found via a kernel search and for the parameters C=18, epsilon=0.015 and gamma=0.12, the following results were achieved:

```
Best R2 score: 0.97648 (C=18, epsilon=0.015, gamma=0.12)
Best MSE score: 0.02383 (C=18, epsilon=0.015, gamma=0.12)
```

This combination of parameters results in the highest performance for the load forecasting problem.

5 Bonus Task

Code

```
data = pd.read_csv('Data.csv', delimiter=';')

time_features =  data[['day', 'month', 'hours']] # Only using time-related columns from the CSV
print(time_features)

# Day of the week for the first day of the year 2019 (Tuesday) is day 1 (too extra??)
time_features['day_of_week'] = (time_features['day'] + 1) % 7

time_features['is_weekend'] = np.where(time_features['day_of_week'] >= 5, 1, 0)
time_features['is_weekday'] = np.where(time_features['day_of_week'] < 5, 1, 0)

time_features['is_winter'] = np.where(time_features['month'].isin([12, 1, 2]), 1, 0)
time_features['is_spring'] = np.where(time_features['month'].isin([3, 4, 5]), 1, 0)
time_features['is_summer'] = np.where(time_features['month'].isin([6, 7, 8]), 1, 0)
time_features['is_fall'] = np.where(time_features['month'].isin([9, 10, 11]), 1, 0)

time_features['is_day'] = np.where((time_features['hours'] >= 7) & (time_features['hours'] <= 16), 1, 0)
time_features['is_peak'] = np.where((time_features['hours'] >= 17) & (time_features['hours'] <= 21), 1, 0)
time_features['is_night'] = np.where((time_features['hours'] >= 22) | (time_features['hours'] <= 6), 1, 0)

time_features_binary = time_features.drop(['day', 'month', 'hours','day_of_week'], axis=1)
time_features_binary = time_features_binary[168:]
print(time_features_binary.shape)

print(time_features_binary.head(25))
```

```
y_hourly = data['Actual load (MW)'].values  # Hourly consumption data
y_hourly = y_hourly[168:]

y_day = y_hourly.reshape(-1, 24).sum(axis=1)  # total daily consumption
print(y_day.shape)

X_bonus_daily = time_features_binary.values.reshape(-1, 24, time_features_binary.shape[1])[:, 0, :]

X_train, X_test, y_train, y_test = train_test_split(X_bonus_daily, y_day, test_size=0.25, random_state=42)
```

```

regressor = SVR(kernel='rbf', C=16, epsilon=0.015, gamma=0.12) #best parameters
regressor.fit(X_train, y_train)

y_pred = regressor.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f'Mean Squared Error: {mse:.4f}')
print(f'R2 Score: {r2:.4f}')
```

Result

```
Mean Squared Error: 924318588.9644
R2 Score: -0.0307
```

Probably a wrong implementation since a negative R^2 score indicates that the chosen model does not follow the trend of the data.